PRD: Mock Interview Feature for elevAIte

Version 1.0 · Prepared as a senior developer's build spec · Status: Ready for MVP build

1. Overview

Mock Interview is a feature of the elevAIte website, not a standalone app. It sits in the left sidebar under Navigation, between Resume Analyzer and Skill Gap Analysis. It is a practice tool built only for students. The student picks their **course**, then the **engineer role** they are preparing for (AI Engineer, Full Stack Engineer, etc.), and gets a realistic, timed interview with AI-generated questions and instant feedback on every answer.

The current UI (see screenshot) has five interview types: Technical, HR, Behavioral, System Design, Coding. **All five are removed.** They are replaced by one flow: **Student Interview → Course → Engineer Type → Start**.

Where it fits in elevAIte

Sidebar module	Relationship to Mock Interview
Dashboard	Shows latest interview scores, streak and recent sessions
Resume Analyzer	Later: generate questions from the student's analyzed resume
Skill Gap Analysis	Weak topics from interviews feed the student's skill gaps
Learning Roadmap	Weak topics suggest what to learn next
My Roadmaps	Student can save a roadmap built from interview results
Pricing	Free plan gets limited sessions; paid plans get more

2. Problem Statement

Students preparing for campus placements and internships have no safe place to practice under pressure. Generic question lists on the internet are not tailored to their course or target role, give no feedback, and do not simulate time pressure.

3. Goals and Non-Goals

Goals

- Let a student start a role-specific mock interview in under 30 seconds.
- Generate questions that match the chosen course and engineer type.
- Enforce a visible countdown timer per question to simulate pressure.
- Give a score and actionable feedback after each answer and a summary at the end.
- Let students track improvement over time.

Non-Goals (for v1)

- HR, Behavioral, System Design and live Coding rounds (explicitly removed).
- Video or audio interviews.
- Recruiter or company-side features.
- Building login, payments, pricing or the sidebar; elevAIte already owns these.

4. Target Users

Persona	Description	Need
Final-year B.Tech student	Preparing for campus placements	Role-specific practice with feedback
Pre-final-year student	Preparing for internships	Confidence and basics
BCA / MCA / B.Sc CS student	Targeting software or data roles	Same practice without a B.Tech-only bias

5. Core User Flow

1. Student opens the app and sees the landing screen (same design as the screenshot, with the new selectors).
2. Student chooses **Course** from a dropdown.
3. Student chooses **Engineer Type** from a second dropdown. The options depend on the course.
4. Student chooses **Difficulty** (Beginner, Intermediate, Advanced) and **Number of Questions** (5, 10, 15).
5. Student clicks **Start Interview**.
6. The right panel shows one question at a time with a countdown timer.

7. Student types an answer and submits, or the timer runs out and the answer auto-submits.
8. AI scores the answer and shows short feedback plus an ideal answer outline.
9. After the last question, a **Result Summary** appears: overall score, strengths, weak topics, and next steps.
10. The session is saved to history.

6. Selection Options

6.1 Course

- B.Tech / B.E. (CSE, IT, AI & ML, Data Science, ECE)
- BCA
- MCA
- B.Sc (CS / IT)
- M.Tech
- Diploma (CS / IT)

6.2 Engineer Type

Engineer Type	Core Topics Covered
AI Engineer	ML basics, deep learning, LLMs, prompt engineering, RAG, model deployment
Machine Learning Engineer	Algorithms, feature engineering, evaluation metrics, MLOps basics
Data Scientist / Analyst	Statistics, SQL, Python, pandas, visualization, case thinking
Full Stack Engineer	HTML, CSS, JavaScript, React, Node.js, REST APIs, databases
Frontend Engineer	JavaScript, React, CSS, accessibility, performance, browser basics
Backend Engineer	APIs, databases, authentication, caching, scalability basics
Mobile App Engineer	Android or iOS basics, Flutter or React Native, app lifecycle

Engineer Type	Core Topics Covered
DevOps / Cloud Engineer	Linux, Git, Docker, CI/CD, AWS or Azure basics
Cybersecurity Engineer	Networking, OWASP Top 10, cryptography basics, threat thinking
QA / Test Engineer	Testing types, automation, test cases, bug reporting
Embedded / IoT Engineer	C/C++, microcontrollers, protocols, RTOS basics

The course controls which engineer types are shown. Example: B.Tech AI & ML shows AI Engineer and ML Engineer first. The list is stored in a config file so new roles can be added without code changes.

7. Functional Requirements

ID	Requirement	Priority
FR-1	Show only the Student Interview flow; no HR, Behavioral, System Design or Coding types	Must
FR-2	Course dropdown with the options in section 6.1	Must
FR-3	Engineer Type dropdown that updates based on the selected course	Must
FR-4	Start button is disabled until course and engineer type are chosen	Must
FR-5	Generate questions using the Claude API based on course, role, difficulty and count	Must
FR-6	Countdown timer per question (default 3:00 as in the UI), turns red under 30 seconds, auto-submits at 0	Must
FR-7	Text answer box with character count	Must
FR-8	AI evaluation of each answer: score out of 10, what was good, what was missing, short ideal answer	Must
FR-9	Result summary screen with overall score and weak topics	Must
FR-10	Fallback question bank (static JSON) used if the API fails	Must

ID	Requirement	Priority
FR-11	Save session history to the signed-in student's elevAIte account	Should
FR-12	Show latest score and streak on the elevAIte Dashboard	Should
FR-13	Skip question and Pause interview	Should
FR-14	Voice answer with speech-to-text	Could (v2)
FR-15	Download result as PDF	Could (v2)

Integration with the elevAIte website

ID	Requirement	Priority
IR-1	Page opens from the Mock Interview sidebar item, which shows the active (highlighted) state	Must
IR-2	Only signed-in students can use it; reuse the site's existing auth session	Must
IR-3	Reuse the site's layout, fonts, colors and shared components; do not rebuild the sidebar or header	Must
IR-4	Prefill course from the student's elevAIte profile when available	Should
IR-5	Enforce plan limits from Pricing (for example free sessions per month); exact limits to be decided	Must
IR-6	Save results so Dashboard, Skill Gap Analysis and Learning Roadmap can read them	Should

8. UI Specification

Keep the existing visual style: soft blue gradient background, white rounded cards, indigo accent text, and the purple pill button.

Placement: The feature renders in elevAIte's main content area, to the right of the sidebar. The sidebar, logo and navigation belong to the site shell and are not part of this feature.

Header: small label "STUDENT INTERVIEW", heading "Practice under realistic pressure and improve fast", timer pill on the right.

Left card (Setup): title "Set up your interview", Course dropdown, Engineer Type dropdown, Difficulty selector, Question count selector, **Start Interview** button.

Right card (Interview Panel) has four states:

1. **Empty:** "Select a course and role, then start your first interview." (dashed box as in the screenshot)
2. **Question:** progress label (Question 3 of 10), question text, answer textarea, Submit and Skip buttons.
3. **Feedback:** score, strengths, gaps, ideal answer, Next button.
4. **Summary:** overall score, per-question scores, weak topics, Retry and New Interview buttons.

The layout must be responsive: on mobile, the two cards stack vertically with setup on top.

9. Recommended Tech Stack

The feature must follow elevAIte's existing stack. Check the current codebase first and match its framework, styling and auth. The table below is the default if the site has no decision yet.

Layer	Choice	Reason
Frontend	React + Vite + Tailwind CSS	Fast, simple, great AI-assisted coding support
Backend	Node.js + Express (or Vercel serverless functions)	Keeps the API key hidden on the server
AI	Claude API (Anthropic)	Question generation and answer evaluation
Storage (MVP)	localStorage	No database setup needed
Storage (v2)	Supabase (Postgres + Auth)	Free tier, easy login
Hosting	Vercel (frontend + API)	Free, deploys from GitHub

Important: never call the Claude API directly from the browser with your key. Always go through your own backend route.

10. Architecture

- The React app sends the setup choices to the backend.
- The backend builds a prompt, calls the Claude API, validates the JSON, and returns it.
- The frontend runs the timer and interview state locally.
- Evaluation is a second backend call per answer.

Inside elevAIte, the backend routes live in the site's existing API, and each route checks the student's session before running.

API Endpoints

Method	Endpoint	Purpose
POST	/api/questions	Generate questions for course, role, difficulty, count
POST	/api/evaluate	Score one answer and return feedback
POST	/api/summary	Build the final summary from all results (optional; can be computed client-side)

Example: /api/questions request

```
{
  "course": "B.Tech CSE",
  "role": "AI Engineer",
  "difficulty": "Intermediate",
  "count": 10
}
```

Example: /api/questions response

```
{
  "questions": [
    {
      "id": 1,
      "text": "Explain the difference between overfitting and underfitting.",
      "topic": "ML Basics",
      "timeLimitSec": 180
    }
  ]
}
```

Example: /api/evaluate response

```
{
  "score": 7,
  "strengths": ["Clear definition of overfitting"],
  "gaps": ["No mention of regularization or cross-validation"],
  "idealAnswer": "Short model answer here.",
  "topic": "ML Basics"
}
```

11. Data Model

Session

- id, userId (from the elevAIte account), createdAt, course, role, difficulty, questionCount
- answers: list of { questionId, questionText, topic, answerText, score, strengths, gaps, timeTakenSec }
- overallScore, weakTopics

Config (roles.json)

- course name → list of allowed engineer types
- engineer type → topic list used in the prompt

12. AI Prompt Design

Question generation (system prompt idea): "You are a senior interviewer conducting a campus interview for a {course} student applying as a {role}. Generate {count} {difficulty}-level questions that fresher students are realistically asked. Mix concept, practical and scenario questions. Return only valid JSON in the given schema."

Evaluation (system prompt idea): "You are a strict but fair interviewer. Score the student's answer from 0 to 10, list strengths and gaps, and give a short ideal answer. Be encouraging and specific. Return only valid JSON."

Rules for both prompts: force JSON-only output, validate the response on the server, and retry once if parsing fails.

13. Non-Functional Requirements

- **Performance:** questions load in under 5 seconds; evaluation returns in under 4 seconds.

- **Reliability:** if the API fails, fall back to the static question bank and show a friendly message.
- **Security:** API key stored only in environment variables; rate limit the backend (for example 20 requests per minute per IP); never log full answers with personal data.
- **Accessibility:** keyboard navigation, visible focus states, sufficient color contrast, labels on all form fields.
- **Cost control:** cap answer length (for example 2000 characters), cap questions per session, use a lower-cost Claude model for evaluation.

14. Success Metrics

- Students who finish a full session (target: above 60 percent).
- Average sessions per student per week.
- Score improvement between a student's first and fifth session.
- API error rate below 2 percent.

15. Milestones

Phase	Scope	Estimate
1	Add the Mock Interview page inside elevAIte, connect the sidebar route, build the setup card with course and role dropdowns	1 day
2	Interview panel states, timer, answer submission	2 days
3	Backend routes, Claude API integration, fallback question bank	2 days
4	Evaluation, feedback screen, summary screen	2 days
5	localStorage history, responsive polish, testing	2 days
6	Release behind a feature flag, test with a small group of students, then launch to everyone	1 day

16. Acceptance Criteria (MVP)

- Only the Student Interview flow exists; no HR, Behavioral, System Design or Coding options anywhere.
- A student can pick course and engineer type, and the Start button enables only then.

- Questions shown match the chosen role (an AI Engineer never sees only frontend questions).
- The timer counts down, warns near zero, and auto-submits.
- Every answer receives a score and feedback.
- The summary shows overall score and weak topics.
- The feature works on mobile and desktop inside the elevAIte layout and opens from the Mock Interview sidebar item.
- The API key is not visible in the browser network tab or source.

17. Risks and Mitigations

Risk	Mitigation
AI returns invalid JSON	Server validation, one retry, then fallback
Weak or generic questions	Role-specific topic lists in config and a clear difficulty rubric in the prompt
High API cost	Question and answer limits, rate limiting, cheaper model for evaluation
Students gaming the system with copy-paste	Show time taken; v2 can add paste detection
API key leak	Backend proxy only, environment variables, never commit .env

18. Future Scope (v2 and beyond)

- Progress charts on the Dashboard.
- Voice-based answers and speech-to-text.
- Company-specific tracks (for example product companies and service companies).
- Resume-based question generation using Resume Analyzer data.
- Leaderboard and streaks for motivation.
- Coding and system design rounds, added later as separate modules.

19. Open Questions

- Should the timer be one total timer or per question? (This PRD assumes per question, 3:00.)

- Should students be able to retake the same question set?
- Should the app support languages other than English?
- What is elevAIte's current frontend stack and auth provider?
- What are the free and paid plan limits for mock interviews?